prompts.md — AI Job Salary Classification Project

Documented prompts used to build `AI_Job_Salary_Classification_Project.ipynb`, following the **4-element prompt structure** (Instruction · Context · Input · Output) and the **5-element coding-prompt formula** (Goal · Inputs · Outputs · Constraints · Language/Level) from *Lecture 3 — Prompt Engineering*, applied here to a **Supervised Learning — Classification** project (Lecture 6), reusing the same dataset as the Regression project (Lecture 4-5).

Every output below was **run in Colab, read, and verified** before being kept, per the lecture's "Run it · read it · verify it" rule — including catching and fixing a real bug along the way (see the "Debugging Log" section at the end).

Prompt 1 — Project setup (single combined prompt)

Type: Instruction | **Technique:** Explicit constraints, ordered steps (5-element coding formula)

> **Goal:** Build one complete, well-organized classification project in a single response — no

> duplicate cells, no repeated steps.
> **Inputs:** `ai_job_dataset-selected-columns.csv` (15,000 rows: job_id, job_title, salary_usd, salary_currency, experience_level, employment_type, company_location, company_size, employee_residence, remote_ratio).
> **Outputs:** A project title cell, an Executive Summary, imports, data loading + exploration, a derived binary target (`high_salary`), creative EDA, leakage-safe preprocessing, three trained models (Logistic Regression, KNN, Decision Tree), full evaluation (comparison table, classification report, confusion matrix, ROC curve, Precision-Recall curve, feature importance), 5-fold cross-validation, GridSearchCV tuning, model saving, and a closing summary.
> **Constraints:** Beginner-friendly comments above every step; generate everything as ONE clean set of cells.
> **Language/Level:** Python 3 / scikit-learn, Google Colab.

Verify: ran every cell top to bottom; confirmed no duplicate cells and no errors.

Prompt 2 — Create the binary target

Type: Instruction | **Technique:** Zero-shot, direct

> **Instruction:** Create a new binary column `high_salary`: 1 if `salary_usd` is above the median, 0 otherwise.
> **Context:** Turns the original regression target into a

> classification target.
> **Output:** Print the class balance to confirm it's close to 50/50.

Verify: class balance came out exactly 50/50 (7,500 / 7,500) — an ideal, non-imbalanced setup.

Prompt 3 — Creative EDA

Type: Instruction | **Technique:** Descriptive, presentation-ready output

> **Instruction:** Create colorful, presentation-ready charts explaining what drives high salary.
> **Output:** Pie chart of the `high_salary` split, violin plot of `salary_usd` by `experience_level` colored by `high_salary`, boxplot by `company_size`, horizontal bar charts of high-salary rate by top job titles and top countries, and a correlation heatmap.

Verify: all 6 charts rendered with titles/labels; visually the `experience_level` violin plot already hinted at what later became the top feature in the model's coefficients.

Prompt 4 — Leakage-safe preprocessing

Type: Instruction | **Technique:** Explicit constraints (5-element coding formula) — the most important prompt in the project for correctness

> **Goal:** Preprocess the data for classification without leaking the answer into the features.
> **Constraints:** Drop `job_id`, `salary_currency`, **and** `salary_usd` from the features — `salary_usd` must be removed because `high_salary` was derived directly from it; keeping it would let the model "cheat." One-Hot Encode remaining categoricals. Split 80/20 (`random_state=42`) **before** scaling. Fit `StandardScaler` on the training set only.
> **Output:** `X_train`, `X_test`, `y_train`, `y_test`, ready for modeling.

Verify: re-checked that `salary_usd` did not appear in `X.columns` before training — this is exactly the check the lecture calls "Dropout is the target, not an input" (adapted here to salary).

Prompt 5 — Preprocessing evidence

Type: Query | **Technique:** Give it an out avoided — ask for concrete proof, not a claim

> **Instruction:** Print a clear "before vs after" summary as evidence the preprocessing worked: shape before vs after encoding, a table mapping each original column to what was done to it (dropped / scaled / one-hot encoded), and confirmation of the train/test split sizes.

Verify: the shape table, column-action table, and split-size table all matched expectations
(15,000 rows → correctly reduced feature set → 12,000/3,000 train/test split).

Prompt 6 — Train three models

Type: Comparative | **Technique:** Role ("act as an ML engineer comparing options")

> **Instruction:** Train Logistic Regression, K-Nearest Neighbors, and Decision Tree Classifier
> on the same preprocessed training data.

Verify: all three trained without errors on the corrected (real) dataset.

Prompt 7 — Evaluate & compare all three models

Type: Evaluative | **Technique:** Descriptive output format

> **Instruction:** Put all three models' accuracy, precision, recall, and F1 score into one
> comparison table, highlighting the best score in each column.

Verify (real numbers):
Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9227	0.9066	0.9428	0.9243
KNN	0.8787	0.8696	0.8916	0.8804
Decision Tree	0.8987	0.9037	0.8929	0.8983

Logistic Regression won on every metric — read and confirmed directly from the table.

Prompt 8 — Logistic Regression deep dive

Type: Evaluative | **Technique:** Chain-of-Thought (multiple linked diagnostics in one pass)

> **Instruction:** For Logistic Regression specifically, show the full `classification_report`, a styled confusion matrix heatmap, an ROC curve with AUC, a Precision-Recall curve, and a horizontal bar chart of the top 10 most influential features from the model's coefficients.

Verify: confusion matrix counts matched the precision/recall numbers printed above it (cross-checked manually); top features were `experience_level_EX` and `experience_level_SE`, consistent with the EDA violin plot from Prompt 3.

Prompt 9 — Cross-validation

Type: Instruction | **Technique:** Zero-shot

> **Instruction:** Run 5-fold cross-validation on Logistic Regression and print the mean and standard deviation of the accuracy scores.

Verify: cross-validated accuracy was close to the single test-set accuracy — evidence the model generalizes rather than overfitting one split.

Prompt 10 — Hyperparameter tuning (GridSearchCV)

Type: Instruction | **Technique:** Explicit constraints (5-element coding formula)

> **Goal:** Try to improve Logistic Regression's accuracy via hyperparameter search.
> **Constraints:** Search `C` in `[0.01, 0.1, 1, 10, 100]` and `penalty` in `[l1, l2]` with `solver="liblinear"`, using 5-fold CV. Compare tuned vs. original metrics side by side.

Verify: best params were `C=1, penalty='l1'`; improvement was marginal (92.13% → ~92.2%) — read as a sign the untuned model was already near-optimal, not a failed search.

Prompt 11 — Save the model

Type: Instruction | **Technique:** Zero-shot

> **Instruction:** Save the best-performing model with `joblib.dump()` as `best_model.pkl` and print a confirmation message.

Verify: confirmation message printed; file appeared in the Colab file browser.

Prompt 12 — Fill in the real results (auto-summary)

Type: Instruction | **Technique:** Explicit constraint — "use actual output, not placeholders"

> **Instruction:** Update the Executive Summary's "Key Result" line and the Closing Summary's bracketed placeholders using the **actual** values from the cells above (best model, its accuracy/F1, whether tuning helped, and the top influential features) — not example text.

Verify: read the final Executive Summary and Closing Summary myself and confirmed every number (92.3% accuracy, `experience_level_EX` / `EN` as top features) matched the real tables and charts earlier in the notebook, rather than trusting the AI's wording blindly.

Hands-on Task — Fix the Prompt

Lazy prompt	Rewritten prompt	Type
"make it classify"	"Train a Logistic Regression classifier on `X_train_scaled` / `y_train` and return the fitted model. Beginner-friendly comments."	Instruction

Lazy prompt	Rewritten prompt	Type
"check accuracy"	"Evaluate the model on `X_test` / `y_test`: print accuracy, precision, recall, and F1 score."	Instruction
"why is it guessing randomly"	"My model's accuracy is 51%, close to random guessing on a balanced dataset. Reason step by step: check the loaded data's summary stats, then the target creation, then the preprocessing, to find where the signal was lost."	Query + Chain-of-Thought
"make it better"	"Use GridSearchCV to tune Logistic Regression's `C` and `penalty` with 5-fold CV, and report whether it actually improved test-set metrics."	Instruction

Debugging Log — Run, Read & Verify in practice

This project hit a real bug that's worth documenting, since catching it *is* the "Read & Verify" skill from the lecture in action:

1. **Symptom:** Logistic Regression accuracy came back at ~51% — essentially random on a balanced 50/50 target.

2. **Investigation:** Instead of accepting the number, compared the printed `median_salary` ($120,447) against the value expected from the real dataset (~$99,705), and checked `job_id` — it showed plain integers (0, 1, 2...) instead of the real format (`AI00001`, `AI00002` ...).
3. **Root cause:** The file Colab was actually reading was a **synthetic/placeholder CSV** that Gemini had generated earlier as a fallback ("if file not found, create random data") — not the real uploaded dataset. Random data has no real relationship between features and target, which is exactly why the model performed at chance level.
4. **Fix:** Removed the placeholder file, re-uploaded the real CSV, and corrected the filename in the `pd.read_csv()` call.
5. **Confirmation:** Re-ran the full notebook — `job_id` showed `AI00001` format, median salary matched $99,705, and Logistic Regression accuracy returned to 92.3%.

Lesson: a plausible-looking number (51%) can still be wrong; verifying it against an independent fact (the known median salary) is what caught a bug that reading the code alone would have missed.